

# How We Finally Got the Wake Back

A plain-language guide to the R9 trust-street implementation  
(32 pressure taps + physics, nothing else)

ModalPINN project — R9

August 2026

## Contents

|          |                                                                |          |
|----------|----------------------------------------------------------------|----------|
| <b>1</b> | <b>The problem, in one picture</b>                             | <b>2</b> |
| <b>2</b> | <b>Why it kept failing (runs R1–R8)</b>                        | <b>2</b> |
| <b>3</b> | <b>Step 1: squeeze everything out of the taps</b>              | <b>3</b> |
| <b>4</b> | <b>Step 2: build the sketch — a vortex street from 1911</b>    | <b>3</b> |
| <b>5</b> | <b>Step 3: the cage — a trust region the wake can’t escape</b> | <b>4</b> |
| <b>6</b> | <b>Did it work? (numbers, honestly)</b>                        | <b>5</b> |
| <b>7</b> | <b>What’s actually in the code (file by file)</b>              | <b>6</b> |
| 7.1      | R9/src/street_prior.py — taps → sketch . . . . .               | 6        |
| 7.2      | R9/src/NN_functions.py — the sketch as graph ops . . . . .     | 6        |
| 7.3      | R9/src/ModalPINN_VortexShedding.py — four new flags . . . . .  | 6        |
| 7.4      | R9/src/evaluate_regions.py — honest scoring . . . . .          | 6        |
| 7.5      | The Colab notebook — same discipline as R7/R8 . . . . .        | 6        |
| <b>8</b> | <b>The one-paragraph version</b>                               | <b>7</b> |

# 1 The problem, in one picture

You have a cylinder in a flow at  $Re = 100$ . Behind it, vortices peel off alternately from the top and bottom — the Kármán vortex street. You are allowed to measure *only* the pressure at 32 points on the cylinder’s surface. From those 32 wiggling signals, you want to reconstruct the entire unsteady flow field: velocity and pressure, everywhere, at every time.

**Analogy.** Imagine standing with your back to the ocean, feeling the water pressure on your legs, and trying to draw every wave behind you. Sounds impossible — except waves follow rules (physics), and the pattern of pushes on your legs carries more information than you’d think.

The tool for this was ModalPINN: a neural network that writes each flow quantity as a Fourier series in time,

$$q(x, y, t) = q_0(x, y) + \sum_{k=1}^3 2 \operatorname{Re} \left[ q_k(x, y) e^{ik\omega_0 t} \right], \quad (1)$$

so the network only has to learn *spatial maps*  $q_k(x, y)$  (one “mode shape” per harmonic  $k$ ), and time comes for free through the known shedding frequency  $\omega_0$ . Training minimizes two things: the mismatch at the 32 taps, and the Navier–Stokes residual at thousands of random “collocation” points in the domain.

## 2 Why it kept failing (runs R1–R8)

Every attempt produced the same picture: a beautiful oscillation right at the cylinder... that fades to nothing within one or two diameters downstream. A dead wake. Eight rounds of fixes — extra loss terms, boundary priors, symmetry, warm starts, causal weighting — and the wake stayed dead.

In this project we finally *measured* why, and it turned out the optimizer was never broken. It was doing its job perfectly.

### The trap (the key insight of the whole project)

At  $Re = 100$  the *steady* flow — cylinder with a long, calm recirculation bubble and **zero** oscillation — is an *exact* solution of the Navier–Stokes equations. In a real flow it’s unstable and nature never keeps it; but a PINN doesn’t march in time, it only checks “do the equations balance?” For the dead wake, they balance *perfectly*. So “no wake” has zero physics penalty, and only the 32 taps — which sit on the cylinder, far from the wake — object.

**Analogy.** You ask an artist to paint “a stormy sea,” but you grade the painting only with a ruler held at the frame’s edge (the taps) and a checklist that flat, calm water also satisfies (the equations). The artist quickly learns that painting a flat sea near the middle is easier and loses no points. Your artist isn’t lazy — your grading scheme literally prefers the calm sea.

We quantified this: killing the wake changes the oscillating-mode physics residual by only a factor of  $\sim 5$  (peanuts compared to other loss terms), and the fully trained dead solution scores about **430× better** than the true field under the old objective. The dead wake isn’t a failure to find the answer; it’s a *better answer* under the old rules.

We also tested the obvious repair — write better rules. Four new loss terms (amplitude-normalized residuals, mean-flow eigenmode consistency, momentum budgets anchored to the measured lift, hard symmetry) all flip the ranking so the true field wins on paper. Trained, every one of them still died. Gradient descent starts near zero oscillation, and the road from “tiny wake” to “real wake” goes *uphill* before it goes down. Better rules don’t help if the contestant can still take the easy exit.

#### The R9 idea in one sentence

Stop trying to make the optimizer *prefer* a live wake — **make a dead wake impossible to express**, then let the optimizer merely *correct* a physically sensible starting sketch.

### 3 Step 1: squeeze everything out of the taps

Before any neural network, the 32 tap signals alone hand us a surprising amount — this is the “free information” step (`street_prior.py`, pure numpy, runs in a minute):

- **The clock**  $\omega_0$ : fit a sinusoid to the lift force (integrate tap pressure around the cylinder)  $\Rightarrow \omega_0 = 1.0357$  — within 0.02% of the value the project always used.
- **The drag**  $C_D$ : same integration, streamwise direction  $\Rightarrow$  pressure drag 0.989.
- **The lift oscillation**  $|L_1| = 0.143$ , *with its phase* — this tells us the wake genuinely oscillates (a dead wake would show  $|L_1| \approx 0$ ), and *when* each vortex leaves.
- **32 little Fourier series**: each tap’s pressure decomposed into mean + harmonics  $k = 1, 2, 3$  — the exact targets the network must hit at the wall.

**Analogy.** It’s forensic work: from the rhythm of the pushes on your legs you get the waves’ tempo; from their average strength, how much energy the sea carries; from the pattern around your legs, which direction the swell comes from. No camera needed — just physics on the signals you have.

### 4 Step 2: build the sketch — a vortex street from 1911

Here’s the creative core. Von Kármán solved the idealized vortex street analytically over a century ago: two staggered rows of point vortices, and *closed-form formulas* connecting spacing, strength, drag and speed. No simulation, no PDE solver — formulas. We use three of them, each fed by a tap-measured number:

1. **Spacing from the clock.** Vortices drift downstream at speed  $U_c$  and pass any point at the shedding frequency, so the row spacing is  $a = 2\pi U_c / \omega_0$ .
2. **Self-consistency.** A staggered street swims upstream against the flow at  $\Gamma / (\sqrt{8} a)$ , so  $U_c = 1 - \Gamma / (\sqrt{8} a)$ . Two equations, two unknowns — solved by simple iteration.

3. **Strength from the drag.** Kármán’s drag formula links the vortex circulation  $\Gamma$  to the body’s drag. We measured the drag at the taps, so we invert the formula:  $\Gamma = 2.53$ .

Then three finishing touches make it a *fluid* rather than a cartoon:

- **Soft cores** (Lamb–Oseen): real vortices aren’t points; their cores also *grow* downstream by ordinary viscous diffusion,  $r_c^2(x) = r_0^2 + 4\nu(x - x_f)/U_c$ . That’s why the far wake gently fades.
- **Image vortices**: for every street vortex we place a mirror vortex inside the cylinder (the Milne–Thomson trick), which forces the flow to slide *around* the cylinder instead of through it.
- **A formation ramp**: the street doesn’t exist at the cylinder face; it switches on smoothly around the formation point  $x_f \approx 1.2$ .

#### Why the mirror vortices earn their keep

A bare vortex street can’t tell “vortex leaving the top now” from “vortex leaving the bottom now” — its surface pressure pattern looks the same half a period apart. The mirror vortices make the street’s induced *surface* pressure physically meaningful, so matching it against the measured tap harmonics pins down the **phase**. Without them our first fit was almost exactly half a period off — a live wake, marching perfectly out of step. (Our learned-decoder experiment hit the same wall, which is *why* we know the mirror physics is the load-bearing part.)

How good is the sketch alone — zero training, straight out of the formulas? Far-wake  $v$ -error  $E_v \approx 0.40$  with pattern correlation 0.99 against the reference CFD. Every neural network before this sat at  $E_v \approx 1.0$  back there (1.0 = “might as well predict no wake”). The 1911 formulas, fed by 32 pressure sensors, beat eight rounds of PINNs in the far field. The near field is the sketch’s weak spot (no boundary layer,  $E_v \approx 1.9$  at the cylinder) — and that’s fine, because the near field is exactly what taps + a network already do well.

## 5 Step 3: the cage — a trust region the wake can’t escape

Now the network comes back, but with different job security. Instead of letting it draw the oscillating modes freely,

$$q_k(x, y) = \text{network}(x, y) \quad (\text{old, can output 0 — and will}), \quad (2)$$

each oscillating mode is written as the sketch plus a *bounded* correction:

$$q_k(x, y) = S_k(x, y) + (\rho |S_k(x, y)| + c) \cdot \tanh(\text{network}(x, y)) \quad k = 1, 2, 3, \quad (3)$$

with  $\rho = 0.6$  and a small floor  $c$ . Because  $\tanh$  can never exceed  $\pm 1$ , the network can reshape, re-phase, and locally fix the street — by up to 60% of its local amplitude — but wherever the sketch is alive, **zero is mathematically outside the reachable set**. The mean flow  $q_0$  stays a

completely free network (the sketch has nothing useful to say about boundary layers; the network and the taps handle that).

**Analogy.** The old setup handed the artist a blank canvas and hoped. The new one hands them your pencil sketch of the storm and a rule: “paint over it as much as 60% — but you may not white it out.” The lazy calm-sea exit is bricked up. The only way to lower the physics penalty now is to make the storm *more correct*.

Training then uses the *plain* original loss — physics residual + tap mismatch + boundary conditions. No exotic terms. Once the search space is right, the old objective was fine all along.

## 6 Did it work? (numbers, honestly)

Local testbed (a faithful PyTorch reimplementation that first *reproduced* the dead-wake failure, so its verdicts transfer):

|                                      | near-wake $E_v$ | far-wake $E_v$ |
|--------------------------------------|-----------------|----------------|
| best previous PINN (R7)              | 0.96            | 1.00           |
| testbed baseline (old loss, control) | 1.00            | 0.99           |
| four “better loss” arms              | 0.99–1.14       | 0.99–1.01      |
| sketch alone (no training)           | 0.95            | 0.40           |
| <b>trust ansatz (R9 mechanism)</b>   | <b>0.64</b>     | <b>0.40</b>    |

Robustness — the whole pipeline re-derived from scratch each time: 1% tap noise  $\rightarrow$  0.38; 5% noise  $\rightarrow$  0.38; only 16 taps  $\rightarrow$  0.38; different network seeds  $\rightarrow$  0.376–0.384 (far-wake  $E_v$ ). The result doesn’t hinge on a lucky configuration.

### Honest limitations (dissertation-ready caveats)

- Far-wake *pressure* is still poor — velocity is the deliverable.
- Near-cylinder mean flow is worse than R7’s warm-started fit; R9 deliberately skips the warm start so credit is unambiguous. The two are composable (planned R9b: warm-start  $q_0$  only).
- The ansatz *assumes* a periodic street exists. Justified here because the taps show a clean single-frequency lift oscillation — but it’s a prior, and it would be the wrong prior for a non-shedding flow.
- Development used validation comparisons to steer choices (as every benchmark method does); the pre-registered test on a new case (e.g.  $Re = 120$ – $150$ ) is the natural follow-up.

## 7 What’s actually in the code (file by file)

### 7.1 R9/src/street\_prior.py — taps $\rightarrow$ sketch

Numpy only,  $\sim 1$  minute. Reads the dataset *only* to pull the same 32 tap signals the trainer uses. Fits  $\omega_0$ , integrates forces, solves the Kármán relations, matches the mirror-street’s surface pressure to the measured tap harmonics (that pins  $x_f$ ,  $r_0$ , and the phase), and — because the exact vortex sums are awkward as TF1 graph ops — calibrates a **closed-form single-harmonic version** of the street against the full numeric one (agreement 0.999). Everything lands in `street_prior_Ntap32.npz`: eight numbers and two calibration factors. That tiny file *is* the sketch.

### 7.2 R9/src/NN\_functions.py — the sketch as graph ops

New function `street_modes_k`: evaluates the closed-form street mode  $S_k$  at any  $(x, y)$  as TensorFlow operations (verified against the numpy reference to  $\sim 10^{-6}$ ). The trust wrap lives in `out_nn_modes_uv` / `out_nn_modes_p`: for  $k \geq 1$  it computes  $S_k + (\rho|S_k| + c) \tanh(\cdot)$ ; for  $k = 0$  nothing changes. It composes with all the existing priors (no-slip mask, freestream blending, inlet damping, symmetry) exactly as before.

### 7.3 R9/src/ModalPINN\_VortexShedding.py — four new flags

`--TrustStreet` (switch it on), `--StreetPrior` (the `.npz`), `--TrustRho`, `--TrustCap`. The prior is loaded once as plain constants and threaded through the six wrapper functions every loss and plot already goes through — so no loss term, plot, or checkpoint path can accidentally bypass the ansatz. The run command is deliberately *minimal*: physics + taps + the two standard BC priors. BVF, K0Loss, PhaseLoss: all off. If the wake lives, we know exactly why.

### 7.4 R9/src/evaluate\_regions.py — honest scoring

A model trained with the trust wrap *is* sketch + correction; restore the weights without the sketch and you’d score a different function (the same restore-bug class the project already hit twice with `--HardSym`). The evaluator now rebuilds the full ansatz, and the notebook copies the prior file into the run folder next to the weights so they can never separate.

### 7.5 The Colab notebook — same discipline as R7/R8

GPU check  $\rightarrow$  Drive mount  $\rightarrow$  pinned TF 1.14 environment  $\rightarrow$  write sources  $\rightarrow$  dataset (Drive-cached)  $\rightarrow$  **derive the sketch** (asserts  $\omega_0$  and the 0.999 calibration)  $\rightarrow$  **smoke test** at full point counts with a hard assertion gate (checks the R9 `TrustStreet` prior: line actually printed — proof the ansatz is live — plus finite loss and L-BFGS iterating)  $\rightarrow$  the real  $\sim 9$  h run  $\rightarrow$  results to Drive  $\rightarrow$  regional scoring. If the gate fails, it stops *before* burning 9 hours.

## 8 The one-paragraph version

The old method asked a network to *find* the wake, graded by rules that a dead wake satisfies perfectly — so it returned a dead wake, every time, for eight runs. R9 instead *derives* a live wake from a century-old analytic solution whose every parameter comes from the 32 taps (frequency from the lift rhythm, strength from the measured drag, phase from the mirror-vortex pressure pattern), then confines the network to a bounded correction band around that sketch where “no wake” simply doesn’t exist as an option. The physics loss — unchanged — finally has to do the only thing left: make the wake *right*. Far-wake error drops from  $\approx 100\%$  to  $\approx 40\%$ , and it survives noise, halved sensors, and reseeding.